$$8^{88^{888}} < 124^{48^{1024}} ?$$

Implementation of level-index arithmetic for very large numbers

by Swastik Banerjee

Abstract

*To represent extremely large real numbers, e.g., Tetrational numbers or huge Power Towers, in the ordinary level-index arithmetic format. Floating-point arithmetic is inadequate to represent these kinds of large numbers, as they are much bigger than the largest real number that can be represented in the Wolfram Language, and hence suffers the problem of overflow. This project introduces a new large number format in Mathematica to virtually abolish the problem of overflow, and adds support for this new format in many of the usual mathematical operations such as +, *, ^ etc.*

Try Overflowing Mathematica Ever Again !!

Introduction to Level-Index Arithmetic

In the field of scientific computation generally and especially in experimental computing which is often at the heart of simulation and modeling problems, the availability of a robust system of arithmetic offers many advantages. Many such computational problems are prone to failure due to overflow or underflow or to a lack of advance knowledge of a suitable scaling for the problem.

For example, if we wanted to compute the following, *Mathematica* would throw an *Overflow error:*

```
In[•]:= 8^(88^888) < 128^(48^1024)
```

General: Overflow occurred in computation.

General: Overflow occurred in computation.

```
Out[•]= Overflow[] < Overflow[]
```

The use of a computer arithmetic system which is free of these drawbacks would clearly alleviate any such difficulties.

One such arithmetic is the `Level-index Arithmetic` introduced by *Clenshaw* and *Olver* [1] in 1984.

The idea of the level - index system is to represent a non - negative `real number` ***X*** as

$$X = e^{e^{e^{\cdots^{e^f}}}}$$

where $0 \le f \le 1$ and the process of exponentiation is performed l times, with $l \ge 0$.
l and f are the *level* and *index* of X respectively.

For example,
$X = 1234567 = e^{e^{e^{0.9711308}}}$

Level-index Arithmetic is closed under the four basic arithmetic operations (apart from division by zero, of course) and is therefore free of overflow.The arithmetic system allows very large numbers which may not be representable in a conventional floating-point system to be used during interim computation while still returning meaningful results.

Goal

The goal of my project was to implement Level-index Arithmetic using Mathematica, so that operations involving huge numbers could be done in the future using the Wolfram Language without the general problem of *overflow, e.g.,* an operation like below:

```
In[•]:= LevelIndexArithmetic[8^(88^(888^(50000^(69^(233 89^64^23))))) + 2^(22^(222^(2222^(22 222^(222 222))))) ] // FromLIO
```

```
Out[•]=
```

$$10^{10^{10^{10^{10^{10^{0.209919}}}}}}$$

Research and Implementation

The implementation involved careful study of various research papers written by *Clenshaw*, *Olver*, *Turner* and *Lozier* [2-3] ,and inspecting source codes of other open-source calculators currently in the market that can compute mathematical operations involving fairly large numbers upto a certain degree of precision. However, due to huge approximations made in almost all of the existing implemented algorithms, several issues arose out of them in terms of precision and correct results, and a real need to do a careful implementation of the core algorithms was felt to make the results more accurate.

Functions Implemented

◆ ToLIO[n_*Real*] :

The **ToLIO[n_*Real*]** function takes any *Real Number* as input and converts it into an **LIO[sign, level, index]** object, as per the original Ordinary Level-Index Arithmetic notation of a number, having a *sign*, a

level and an *index.*
Note: Sign is always either +1 or -1.

In[•]:=
```
ToLIO[1 234 567]
```

Out[•]=

```
LIO[1, 3, 0.971131]
```

◆ PowerForm[LIO[sign, level, index]] :

The **PowerForm[LIO[sign,level,index]]** function takes any **LIO[sign, level, index]** object as input and converts it into a nice, readable *level-index format* in *base 10,* so that it looks like a *Power Tower* of 10s with an index at the end.

In[•]:=
```
PowerForm[LIO[1, 14, 0.677]]
```

Out[•]=

$$10^{10^{10^{10^{10^{10^{10^{10^{10^{10^{10^{10^{10^{0.438599}}}}}}}}}}}}}$$

◆ FromLIO[LIO[sign, level, index]] :

The **FromLIO[LIO[sign,level,index]]** function takes any **LIO[sign, level, index]** object as input and converts it into a floating-point number if it does not overflow. If it overflows in normal floating-point notation, the **PowerForm[]** is invoked automatically, and a *level-index format* of the number in *base 10* is returned.

In[•]:=
```
FromLIO[LIO[1, 3, 0.9]]
```

Out[•]=

```
120 592.
```

In[•]:=
```
FromLIO[LIO[1, 7, 0.65]]
```

Out[•]=

$$10^{10^{10^{10^{10^{10^{0.412713}}}}}}$$

◆ LevelIndexArithmetic[*expr_*] :

The **LevelIndexArithmetic[*expr_*]** function takes *any arithmetic expression* as input, converts it to *LIO[sign, level, index] object*, performs desired *arithmetic operations* on them, and then finally returns the result as an *LIO[sign, level, index] object*.

In[•]:=
```
LevelIndexArithmetic[10^(2^(188^(299^86))) + 8^(99^(677^864)) + 7^(29^233)]
```

Out[•]=

```
LIO[1, 6, 0.768246]
```

*Note: It's generally a good idea to **wrap** the **LevelIndexArithmetic[]** function with **FromLIO[]**, to get the final output in a nice, readable format.*

In[•]:=
```
LevelIndexArithmetic[10^2^188^299^86 + 8^99^677^864 + 7^29^233] // FromLIO
```

Out[•]=

$$10^{10^{10^{10^{10^{0.53}}}}}$$

Handling Divisions

Since *Division* operations of two numbers inside *LevelIndexArithmetic[]* function would actually get interpreted as *Multiplication of two LIO[]s with the second one having a negative index*, as since *negative index of LIO[] is not defined* as per the normal level-index notation system, hence it would throw an *error* if Division operation is tried inside one single LevelIndexArithmetic[] function.

The same can be handled in the following manner:

If you want to do $\frac{10^{88^{677^{309}}}}{10^{88^{97}}}$,

simply convert the numerator and the denominator each to LIO[]s first using LevelIndexArithmetic[] function, and then do the Division.

In[•]:=
```
LevelIndexArithmetic[10^88^677^309] / LevelIndexArithmetic[10^88^97] // FromLIO
```

Out[•]=

$$10^{10^{10^{10^{10^{0.46864}}}}}$$

A Look into the Backend

The Addition and Subtraction operations were implemented by forming three *Recursive Functions*, as suggested by the original algorithm in the paper "*Level-Index Arithmetic Operations*" by Clenshaw and Oliver [1]

The Multiplication, Division and Exponentiation operations were implemented using the following rules :

$x * y = e^{\text{Log}[x]+\text{Log}[y]}$

$x / y = e^{\text{Log}[x]-\text{Log}[y]}$

$x^y = e^{\text{Log}[x]*y}$

The Full Code can be seen here :

Show full code

In[•]:=
```
(*-----------subtraction function begins-----------------*)
```

```
subtraction[LIO[1, 0, f_], LIO[1, 0, g_]] := LIO[1, 0, f-g]
subtraction[LIO[1,l_,f_], LIO[1,m_,g_]] := Catch @ Module[{a,b,c},
    a[l-1] = Exp[-f];
    a[j_] := a[j] = Quiet@Check[Exp[-1/a[j+1]], 0];
    If[m==0,
        b[0] = a[0]*g,
        b[m-1] = a[m-1] Exp[g];
        b[j_] := b[j] = Quiet@Check[Exp[-(1-b[j+1])/a[j+1]], 0]
    ];
    c[0] = 1 - b[0];
    c[j_] := c[j] = 1 + a[j] Log[c[j-1]];
    Do[
        If[c[j] < a[j], Throw[LIO[1, j, c[j]/a[j]]]],
        {j, 0, l-1}
    ];
    LIO[1, l, f + Log[c[l-1]]]
];
(*-----------subtraction function ends-----------------*)


(*-----------addition function begins-----------------*)
addition[LIO[1, 0, f_], LIO[1, 0, g_]] := (
    If[f+g<1,LIO[1,0,f+g],
        LIO[1,1,Log[f+g]]
    ]
);
addition[LIO[1,l_,f_], LIO[1,m_,g_]] := Catch @ Module[{a,b,c,hl},
    a[l-1] = Exp[-f];
    a[j_] := a[j] = Quiet@Check[Exp[-1/a[j+1]], 0];
    If[m==0,
        b[0] = a[0] g,
        b[m-1] = a[m-1] Exp[g];
        b[j_] := b[j] = Quiet@Check[Exp[-(1-b[j+1])/a[j+1]], 0]
    ];
    c[0] = 1 + b[0];
    c[j_] := c[j] = 1 + a[j] Log[c[j-1]];

    hl=f + Log[c[l-1]];
    If[hl<1, LIO[1, l, hl],
        LIO[1,l+1,Log[hl]]
    ]
];
(*-----------addition function ends-----------------*)



(*----------------------------------------*)
Sign[LIO[s_,l_,i_]]^:=s                    (*define Sign function for LIO*)

Abs[LIO[s_,l_,i_]]^:=LIO[1,l,i]            (*define Abs function for LIO*)
(*----------------------------------------*)
```

```
a_?NumericQ*LIO[s_,l_,i_]^:=Module[{is=Sign[a]},(*assigning negative signs when "-" is given in
 LIO[is*s,l,i]];


(*------------operations begin-----------------*)
LIO/: LIO[s1_,l_,f_]+LIO[s2_,m_,g_]:=Module[{F,G,sign,res},

    If[l>m,
        F=LIO[s1,l,f];
        G=LIO[s2,m,g],

    If[l<m,
        F=LIO[s2,m,g];                  (*F=Bigger*)
        G=LIO[s1,l,f],                  (*G= Smaller*)
    (*else if l ==m*)
    If[f>g,
        F=LIO[s1,l,f];
        G=LIO[s2,m,g],
    (*if f≤g*)
        F=LIO[s2,m,g];
        G=LIO[s1,l,f]
    ]
    ]
    ];

    sign=Sign[F];                   (*store sign of larger value F*)

    res=If[Sign[F]==Sign[G],
        addition[Abs[F],Abs[G]],
        subtraction[Abs[F],Abs[G]]
    ];

    res[[1]]=sign;  (*set the sign equal to the larger of the two values,i.e, F to the result*)

    res

];
(*------------operations end-----------------*)




(*------------Log and Expo begins-----------------*)

Log[LIO[1,l_,i_]]^:=(
    If[l==0,LIO[-1,0,-Log[i]],LIO[1,l-1,i]]
);

Exp[LIO[s_,l_,i_]]^:=LIO[s,l+1,i]
```

```
Exp[LIO[-1,0,i_]]^:=LIO[1,0,Exp[-i]]
Exp[LIO[-1,l_,i_]]^:=If[LIO[1,l,i]>ToLIO[Log[$MaxNumber]],LIO[1,0,0],LIO[1,0,(1/FromLIO[LIO[1,l
(*-----------Log and Expo ends-----------------*)



(*-----------mul, div and pow begins-------------------*)
LIO/:LIO[s1_,l_,f_]*LIO[s2_,m_,g_]:=(
If[l==0&&f==0 || m==0&&g==0,LIO[1,0,0],
Quiet@Exp[Log[LIO[s1,l,f]]+Log[LIO[s2,m,g]]]
]
)

LIO/:LIO[s1_,l_,f_]/LIO[s2_,m_,g_]:=Quiet@Exp[Log[LIO[s1,l,f]]-Log[LIO[s2,m,g]]]

LIO/:LIO[s1_,l_,f_]^LIO[s2_,m_,g_]:=Quiet@Exp[LIO[s2,m,g]*Log[LIO[s1,l,f]]]

(*-----------mul, div and pow ends-----------------*)


Log10[LIO[1,l_,i_]]^:=Log[LIO[1,l,i]]/LIO[1,1,0.834032445247956`]


(*-----------comparator func begins-----------------*)


LIO[s1_,l_,f_]==LIO[s2_,m_,g_]^:=s1==s2&&l==m&&f==g


LIO[s1_,l_,f_]>LIO[s2_,m_,g_]^:=(
If[s1<0&&s2≥0,Return[False],
    If[s1>0&&s2≤0,Return[True],
        If[s1<0&&s2<0,
            If[l>m,Return[False],
                If[l<m,Return[True],
                    If[f<g,Return[True],Return[False]]
                ]
            ],
        (*If s1≥0 && s2≥0*)
            If[l>m,Return[True],
                If[l<m,Return[False],
                    If[f>g,Return[True],Return[False]]
                ]
            ]
        ]
    ]
]
);


LIO[s1_,l_,f_]≥LIO[s2_,m_,g_]^:=(
```

```
If[LIO[s1,l,f]==LIO[s2,m,g],Return[True],

If[s1<0&&s2≥0,Return[False],
    If[s1>0&&s2≤0,Return[True],
        If[s1<0&&s2<0,
            If[l>m,Return[False],
                If[l<m,Return[True],
                    If[f<g,Return[True],Return[False]]
                ]
            ],
        (*If s1≥0 && s2≥0*)
            If[l>m,Return[True],
                If[l<m,Return[False],
                    If[f>g,Return[True],Return[False]]
                ]
            ]
        ]
    ]
]
]
);


LIO[s1_,l_,f_]<LIO[s2_,m_,g_]^:=(
If[s1<0&&s2≥0,Return[True],
    If[s1>0&&s2≤0,Return[False],
        If[s1<0&&s2<0,
            If[l>m,Return[True],
                If[l<m,Return[False],
                    If[f<g,Return[False],Return[True]]
                ]
            ],
        (*If s1≥0 && s2≥0*)
            If[l>m,Return[False],
                If[l<m,Return[True],
                    If[f>g,Return[False],Return[True]]
                ]
            ]
        ]
    ]
]
);


LIO[s1_,l_,f_]≤LIO[s2_,m_,g_]^:=(
If[LIO[s1,l,f]==LIO[s2,m,g],Return[True],

If[s1<0&&s2≥0,Return[True],
    If[s1>0&&s2≤0,Return[False],
        If[s1<0&&s2<0,
            If[l>m,Return[True],
```

```
                If[l<m,Return[False],
                    If[f<g,Return[False],Return[True]]
                ]
            ],
        (*If s1≥0 && s2≥0*)
            If[l>m,Return[False],
                If[l<m,Return[True],
                    If[f>g,Return[False],Return[True]]
                ]
            ]
        ]
    ]
]
);
(*-----------comparator func ends-----------------*)




(*--------------------final wrap-up functions begin--------------*)

Options[ToLIO]={Precision→MachinePrecision};


ToLIO[n_Real]:=Module[{in=n,l=0},
    While[in≥1,
        in=Log[in];
        l++;
    ];
    LIO[Sign[n],l,in]       (*converts any real to LIO[s,l,f]*)
];


ToLIO[n_?NumericQ,OptionsPattern[]]:=ToLIO[N[n,OptionValue[Precision]]]


PowerForm[LIO[s_,l_,f_]]:=Module[{res=LIO[s,l,f],basecnt=0,ini,i},

    While[res⟦2⟧>0,res=Log10[res];basecnt++];
    ini=Power["10",res⟦3⟧];
    For[i=basecnt-1,i≥1,i--,
        ini=Power["10",ini];
    ];
    ini
];


FromLIO[LIO[s_,l_,f_]]:=Module[{ini,i},
If[l==0,Return[f],
    Quiet@Check[ini=Power[e,f];
```

```
    For[i=l-1,i≥1,i--,
        ini=Power[e,ini]    (*converts LIO[s,l,f] to powertower form with base e*)
    ];
    ini,PowerForm[LIO[s,l,f]]]
    ]
];
FromLIO[x_]:=x     (*for comparisons*)



Options[LevelIndexArithmetic]={Precision→MachinePrecision};
LevelIndexArithmetic[expr_,OptionsPattern[]] := Hold[expr] /. n_?AtomicNumberQ :> ToLIO[N[n,Optio
SetAttributes[LevelIndexArithmetic, HoldFirst];   (*Final function to perform an operation and ge

AtomicNumberQ[x_]:=AtomQ[Unevaluated@x]&&NumericQ[x]
SetAttributes[AtomicNumberQ, HoldFirst];
(*--------------------final wrap-up functions begin--------------*)
```

In[•]:=

A Few More Interesting Operations and Test-Cases

Consistent results for ***Non - Integral*** *Power Towers*

In[•]:=
```
LevelIndexArithmetic[e^(E^(E^(E^Pi)))]//PowerForm
```

Out[•]=

$$10^{10^{10^{10^{10^{0.986219}}}}}$$

In[•]:=
```
LevelIndexArithmetic[10^(2.999999^(3.8764^(8.9913^223.869)))]//FromLIO
```

Out[•]=

$$10^{10^{10^{10^{10^{0.367168}}}}}$$

Comparator Functions

We can now successfully ***compare huge Power Towers*** in *Mathematica* without the problem of over-flow, which might not be possible till date using normal computers or most other calculators so easily.

In[•]:= `LevelIndexArithmetic[8^88^888 < 128^48^1024]`

Out[•]=

```
False
```

Summary

A few functions were successfully fabricated using the Wolfram Language to carry out arithmetic operations of huge numbers in Mathematica using Level-Index Arithmetic, which would otherwise overflow in normal floating-point notation.

Further Works

- Forming more *test cases* and checking the current implementation rigorously for any edge-case.
- Including this in the Wolfram **Function Repository**.
- Extending the implementation to Symmetric Level-Index Arithmetic (SLI).
- Devicing an algorithm to convert huge factorials to Level-Index Objects.
- Extending the functions to Complex realms.

Keywords

- overflow
- level-index
- numerical analysis
- computer arithmetic
- precision
- exponentiation
- Power Towers
- Tetration

Acknowledgment

Mentor: Carl Woll

I would like to thank my mentor for guiding me towards a successful implementation, the friendly and helpful teaching assistants (especially Daniel Sanchez,Jesse Friedman and Philip Maymin) for their

24x7 assist even at all kinds of silly problems I ran into, Stephen Wolfram for his suggestions to take up this project and finally also my friend Nikolay Murzin for his support during the course of the project. They were always eager to help me out whenever I got stuck at any stage.
I would also like to thank my mother for staying up late with me when I worked on hours without sleep on the project on the last few days, and my father for constantly supporting me.

References

- `Hypercalc` by Robert Munafo
- `Level Index Arithmetic Operations` (C. W. Clenshaw and F. W. J. Oliver SIAM Journal on Numerical Analysis Vol. 24, No. 2 (Apr., 1987), pp. 470-485)
- `Tetration` (Wikipedia)
- `Wolfram Language Documentation`

In[ ]:=

1 https://dl.acm.org/doi/10.1145/62.322429

2 https://academic.oup.com/imajna/article-abstract/8/4/517/758814?redirectedFrom=fulltext

3 https://www.jstor.org/stable/2157569?seq=1

1 https://dl.acm.org/doi/10.1145/62.322429